

Code Explanation

Test Data Processing Code Explanation

This code is a test suite written in Pytest for testing data processing functions using PySpark. The tests verify that the functions correctly read and process customer and order data from CSV files.

Overview of the Code

The code consists of test fixtures for creating a SparkSession and generating sample customer and order data in temporary directories. It then tests two data processing functions, `read_customer_data` and `read_order_data`, to ensure they correctly read and process the data.

Step 1: Creating a SparkSession Fixture

The first step is to create a fixture for a SparkSession, which is a fundamental object in PySpark for creating DataFrames and executing Spark operations.

```
@pytest.fixture(scope="module")
def spark():
    return SparkSession.builder
        .appName("TestDataProcessing")
        .master("local[*]")
        .config("spark.sql.warehouse.dir", tempfile.mkdtemp())
        .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
        .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")
        .getOrCreate()
```

Step 2: Generating Sample Data

The next step is to create a fixture that generates sample customer and order data in a temporary directory. This fixture creates CSV files for customer and order data, including some invalid records with null values and duplicates.

```
@pytest.fixture(scope="module")
def sample_data_path():
    temp_dir = tempfile.mkdtemp()
    customer_dir = os.path.join(temp_dir, "customer")
    order_dir = os.path.join(temp_dir, "order")

    os.makedirs(customer_dir)
    os.makedirs(order_dir)

    # Create sample customer data
    with open(os.path.join(customer_dir, "customer.csv"), "w") as f:
        f.write("CustId,Name,EmailId,Regionn")
        f.write("C001,John Doe,john@example.com,Northn")
        f.write("C002,Jane Smith,jane@example.com,Southn")
        f.write("C003,Bob Brown,bob@example.com,Eastn")
        f.write("C004,Alice Green,alice@example.com,Westn")
        f.write("C005,,alice@example.com,Westn") # Invalid record with null name
        f.write("C001,John Doe,john@example.com,Northn") # Duplicate record
```

```

# Create sample order data
with open(os.path.join(order_dir, "order.csv"), "w") as f:
    f.write("OrderId,ItemName,PricePerUnit,Qty,Date,CustIdn")
    f.write("0001,Laptop,1000.0,2,2023-01-15,C001n")
    f.write("0002,Mouse,25.5,4,2023-01-16,C002n")
    f.write("0003,Keyboard,45.75,3,2023-01-17,C003n")
    f.write("0004,Monitor,150.0,1,2023-01-18,C001n")
    f.write("0005,Headphones,,2,2023-01-19,C002n") # Invalid re
cord with null price
    f.write("0001,Laptop,1000.0,2,2023-01-15,C001n") # Duplicat
e record

    yield temp_dir

# Clean up
shutil.rmtree(temp_dir)

```

Step 3: Testing the `read\_customer\_data` Function

The `test\_read\_customer\_data` function tests the `read\_customer\_data` function by verifying that it correctly reads customer data from a CSV file, filters out invalid records, and returns a DataFrame with the expected schema and data.

```

def test_read_customer_data(spark, sample_data_path):
    customer_path = os.path.join(sample_data_path, "customer")
    df = read_customer_data(spark, customer_path)

    assert df.count() == 4

    assert df.schema.names == ["CustId", "Name", "EmailId", "Region"]

    data = df.collect()
    assert any(row.CustId == "C001" and row.Name == "John Doe" for r
ow in data)
    assert any(row.CustId == "C002" and row.Name == "Jane Smith" for
row in data)

    assert not any(row.CustId == "C005" for row in data)

```

Step 4: Testing the `read\_order\_data` Function

The `test\_read\_order\_data` function tests the `read\_order\_data` function by verifying that it correctly reads order data from a CSV file, filters out invalid records, calculates the total amount for each order, and returns a DataFrame with the expected schema and data.

```

def test_read_order_data(spark, sample_data_path):
    order_path = os.path.join(sample_data_path, "order")
    df = read_order_data(spark, order_path)

    assert df.count() == 4

    assert "TotalAmount" in df.schema.names

```

```
data = df.collect()
assert any(row.OrderId == "0001" and row.TotalAmount == 2000.0 f
or row in data)
assert any(row.OrderId == "0002" and row.TotalAmount == 102.0 fo
r row in data)

assert not any(row.OrderId == "0005" for row in data)
```